

5 TOOLS PER TURN COMPLIANCE

Domain: Healthcare

Prompt Lens: Consumer

The first prompt is:

“My allergies have been way worse than usual the past few days - sneezing constantly, nose running even when I am just sitting in class. I am wondering if the air outside has something to do with it since I have been practicing outside for basketball. Can you check what the air quality looks like in Cleveland right now? Also, pull up my allergy condition details from my health records so I can see exactly what I am dealing with.”

This prompt naturally leads to only a very small number of meaningful tool calls:

- one tool call to check current air quality in Cleveland
- one tool call to retrieve the user's allergy condition details
- possibly one additional validating or supporting call depending on the tool design

That means the task is structurally under-scoped from the start. The project guidance requires each prompt to naturally trigger at least 5 tool calls, and each turn should use at least 5 tool calls to handle a multi-step workflow. The same guidance also emphasizes believability, meaning the complexity must arise naturally from the request, not from artificial padding.

Why is this bad

The issue is not just “too few calls.” The real problem is that the prompt does not create enough real work for the agent.

This matters for a few reasons:

1. It does not meet the project's complexity requirement.
The spec doc and the project guidance expect long-horizon, tool-dependent tasks where each turn has enough depth to exercise planning, validation, and grounded reasoning. A prompt that can be solved in 2 to 3 calls does not create that kind of workflow.
2. It makes the turn less realistic for Horus.
The project wants naturally complex prompts, not a simple request stretched into something it was never designed to support. The first prompt is especially important

because it sets up the rest of the task. If the first turn is too narrow, it becomes much harder to build strong later turns without making the conversation feel forced.

3. It weakens the rest of the task design.

A first turn like this leaves very little room for a strong GT, meaningful assistant-turn distribution, or robust rubrics. Even if the contributor writes a technically correct GT, the underlying prompt is still low-value because it did not create enough meaningful complexity in the first place.

How to fix it

The fix is to redesign the prompt so the user's request naturally requires a multi-step workflow with at least 5 meaningful tool calls.

That means:

- combine related sub-needs that genuinely require multiple tools or multiple retrieval steps
- make the user's goal broader in a realistic way, so the model has to gather, compare, verify, and synthesize
- make sure each added part is actually useful to solving the user's problem, not just increasing call count
- avoid trivial "padding" moves like simple calculator calls or unnecessary repeat lookups

A good self-check is:

If I remove one of the tool calls, does the model lose something truly necessary to solve the user's problem?

If the answer is no, the prompt is probably being padded instead of naturally requiring that call.

For a prompt like this, the contributor should have expanded the user's need into a more complete decision-making task. For example, instead of only checking current air quality and allergy records, the user's request would need additional meaningful components such as comparing relevant environmental factors, checking recent exposure conditions, reviewing medication or treatment context from records, and narrowing toward a concrete next-step recommendation grounded in available tools. The key is that each additional step must serve the user's real problem.

GOLDEN TRAJECTORY ISSUES

Case #1: Factual Errors

Domain: Shopping

Prompt Lens: Consumer

Prompt fragment (relevant request):

"I am thinking to felicitate the top three winners with the yoga mat that I ordered in the last 40 days and other participants with flowers because it will be budget friendly. Please check if that yoga mat is in stock or not."

GT first assistant message (Relevant info retrieved from previous turns - assume it's grounded):

As today's date is 2026-02-28 so the last 40 days date range would be from 2026-01-20. I will now check amazon orders history to find out the yoga mat ordered in the last 40 days.

So, last 40 days begin date: 1/20/2026

Relevant tool output fragment:

JSON

```
{
  "order_id": "ORDER-H59V20JT",
  "user_id": "persona_009",
  "status": "delivered",
  "total": 203.98,
  "currency": "USD",
  "created_at": "2026-01-13T22:57:35",
  "shipped_at": "2026-01-16T22:57:35",
  "delivered_at": "2026-01-21T22:57:35",
  "shipping_address": "789 Veterans Blvd, Tampa, FL 33602",
  "items": [
    {
      "id": "oi_141",
      "asin": "B0AMZN1190",
      "quantity": 2,
      "price_at_purchase": 88.0,
      "title": "Lululemon The Reversible Mat 5mm",
      "image_url": "https://amazon.com/images/lulu-reversible.jpg",
      "product_url": "https://amazon.com/dp/B0AMZN1190",
    }
  ]
}
```

```
CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-28T08:08:51.218Z
    "subtotal": 176.0
  },
  {
    "id": "oi_142",
    "asin": "B0BCGHQ34L",
    "quantity": 2,
    "price_at_purchase": 13.99,
    "title": "The Seven Husbands of Evelyn Hugo by Taylor Reid",
    "image_url": "/images/evelyn-hugo.jpg",
    "product_url": "/dp/B0BCGHQ34L",
    "subtotal": 27.98
  }
]
CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-28T08:08:51.218Z
```

Golden Trajectory Final Answer Fragment:

“You ordered Lululemon The Reversible Mat 5mm in the last 40 days and it is in stock right now.”

Problem

The issue is factual drift in the GT. The GT said the yoga mat was purchased within the last 40 days, but the tool output does not support that.

The key date in the tool output is:

“created_at”: “2026-01-13T22:57:35”

If the last 40 days begin on January 20, 2026, then a purchase made on January 13, 2026 falls outside that window.

So the GT is wrong because it treated the yoga mat as a valid match for “the yoga mat that I ordered in the last 40 days” even though the purchase history shows it was ordered before that period.

Probably root cause: the user either miscalculated or did not check the possible outcomes, and the issue remains mainly in the prompt (should be 50 days instead of 40).

How to fix it

The GT should first compare the actual purchase date against the requested time window before deciding that the yoga mat qualifies.

Here, the correct reasoning is:

- the yoga mat appears in the order history
- its purchase date is January 13, 2026
- that date is before January 20, 2026
- therefore it was not ordered within the last 40 days

Because of that, the GT should not proceed as if this is the qualifying yoga mat from the prompt. It should either:

- state that no yoga mat was found within the last 40 days based on the retrieved purchase history, or
- continue searching broader purchase history if the available tool setup supports that

Another option would be to modify the prompt and set the windows to the last 50 days instead.

Good example fixed

A corrected version would say something like:

"I found a yoga mat in your order history: Lululemon The Reversible Mat 5mm. However, it was purchased on January 13, 2026, which is outside the last 40 days if that window starts on January 20, 2026. So this item does not qualify as a yoga mat ordered within the last 40 days."

Another option would be to modify the prompt and set the windows to the last 50 days instead.

The key lesson is simple: before carrying a product forward into the next step, make sure it actually satisfies the prompt's filters exactly as written.

Case #2: Grounding Issues

Domain: Shopping

Prompt Lens: Consumer

The GT says:

"ORDER-SP7DGOGP (cancelled, 2026-01-24): \$4,379.14"

(GT response is much longer, and this can easily be overlooked)

But the tool output only shows:

- `status: "cancelled"`
- `created_at: "2026-01-24T14:28:38"`

It does not show a `cancelled_at` field.

Why is this bad

This is a subtle grounding issue. The response presents `2026-01-24` as if it were the cancellation date, but the tool output only confirms that the order is cancelled and that it was created on that date.

That may seem minor, but this is exactly the kind of small, debatable detail that can lower task quality. The model should not present an inferred date as a confirmed fact.

This is also a good reminder not to trust a long Agent response at a glance. A task can look mostly correct and still fail because of one small statement that goes beyond the tool output.

How to fix it

Do not state `2026-01-24` as the cancellation date unless the tool output explicitly provides that.

Safer alternatives would be:

- "ORDER-SP7DGOGP is the most recent cancelled Amazon order, with a total of \$4,379.14."
- or, if needed, "ORDER-SP7DGOGP was created on 2026-01-24 and is currently marked as cancelled."

The main lesson is:

Check every small factual statement against the tool output. If the data does not explicitly support the exact wording, do not state it as confirmed.

OVERLY PRESCRIPTIVE CRITERIA

Case #1: Query Construction

Domain: Shopping

Prompt Lens: Consumer

Prompt:

"I really need to see my doctor sometime next week, my back has been killing me lately. Can you check next week availability from Monday to Tuesday and let me know if there are any morning slots?"

Also, since she recommended walking daily for my knee, I'm thinking of walking to the appointment to take the chance and get that done for the day. It's about 3 km and, you know, how **Cleveland** weather can be sometimes. Can you check the **weather for the next four days and tell me what to expect on the day with** a morning slot available? Also, once you are there, check for **today's alerts**, wanna be sure there is no risk of visiting my girl later today.

Oh, and I almost forgot, I also need to track today's calories. Get me the **calories for a Grande Iced Chai Tea Latte from Starbucks and a Grilled Chicken Salad from Subway.**"

Rubric sample:

- "The model must pass 'Cleveland' to the 'q' parameter when calling the 'weather-data_weather_alerts' tool to check for today's (20th March, 20206) weather alerts in Cleveland."
- "The model must pass 'Cleveland' to the 'q' parameter when calling the 'weather-data_weather_forecast' tool to retrieve Cleveland's forecast for next week, Monday to Thursday (March 23 to 26, 2026)."
- "The model must pass 'Grilled Chicken Salad Subway' to the 'query' parameter when calling the 'opennutrition_search-food-by-name' tool to find the Subway salad caloric value."
- The model must pass 'Grande Iced Chai Tea Latte Starbucks' to the 'query' parameter when calling the 'opennutrition_search-food-by-name' tool to find the Starbucks drink caloric value.

Why is this bad

This is an overprescription issue. While the user intent refers to Cleveland, enforcing a strict 1:1 mapping is not appropriate, as the location can be expressed in **other equally natural ways when calling 'weather-data_weather_alerts' and 'weather-data_weather_forecast'** (e.g., 'Cleveland, Ohio' or 'Cleveland, OH').

The same applies to “Grilled Chicken Salad Subway” and “Grande Iced Chai Tea Latte Starbucks”. Although this may seem to be a 1:1 mapping between the user intent and the tool reference, the actual item references within the tools are **“Buffalo Chicken with Grilled Chicken Salad by Subway”** and **“Iced Chai Tea Latte, Grande by Starbucks.”**



Since these prompt intents allow for alternatives that remain equally valid and natural while preserving full efficiency, these query constructions should be treated as fuzzy parameters rather than hardcoding them, as outlined in the Horus Rubrics Writing Guidelines.

How to fix it

Write rubrics around must-haves, not around one preferred implementation.

For query construction, check the intent:

- Can Cleveland be expressed in multiple equally efficient ways within the 'weather-data_weather_alerts' and 'weather-data_weather_forecast' tools while remaining naturally aligned with the prompt intent?
 - **Yes:** 'Cleveland, Ohio, USA', 'Cleveland, USA', 'Cleveland, OH '
- Do the food names exactly match the tool outputs as they appear in the prompt?
 - **No:** The prompt intent and the tool output do not have a 1:1 match.

Fixed examples

- “The model must pass a 'q' parameter targeting Cleveland location (e.g., "Cleveland") when calling the 'weather-data_weather_alerts' tool to check for weather alerts in the user's location.”
- “The model must pass a 'q' parameter targeting Cleveland location (e.g., 'Cleveland') when calling the 'weather-data_weather_forecast' tool to retrieve the forecast for next week, Monday to Thursday (March 23 to 26, 2026).”
- “The model must pass a 'query' parameter targeting Grande Iced Chai Tea Latte Starbucks when calling the 'opennutrition_search-food-by-name' tool to find the Starbucks drink caloric information.”
- “The model must pass a 'query' parameter targeting Grilled Chicken Salad Subway when calling the 'opennutrition_search-food-by-name' tool to find the Subway chicken salad caloric value.”

The key lesson is simple: only mirror the Golden Trajectory when the intent explicitly requires it. When multiple valid options exist, rubrics should be written in a way that still correctly evaluates all acceptable trajectories.

Case #2: Outcome

Domain: Shopping

Prompt Lens: Consumer

Prompt:

“One of my apps just exploded in the app store and I got a big bonus because of it. I'd like to build a new PC, and I have decided to spend the money from my most recent cancelled order on Amazon for it. I want both the CPU and the GPU to be from AMD with ratings above 4.7, with the former having an AM5 socket and at least 16 cores, and the latter having between 20 and 24 GB of VRAM. For the PSU, I only want it to be from Corsair and for the price to be in between the two most expensive PSUs I bought last year from the PC store, in 2025. What options are there and how much money is left for the other parts? Everything must be in stock.”

GT response includes:

- budget from the most recent cancelled Amazon order: \$4,379.14

- two most expensive PSUs bought in 2025 from the PC store:

- Dark Power Pro 13 — \$369.99

- Straight Power 12 — \$229.99

Overly prescriptive outcome criteria:

- “The model must report that the budget from the most recent cancelled Amazon order is '\$4,379.14'.”
- “The model must report that one of the two most expensive PSUs bought in 2025 was 'Dark Power Pro 13' with a price of '\$369.99'.”
- “The model must report that one of the two most expensive PSUs bought in 2025 was 'Straight Power 12' with a price of '\$229.99'.”

Why is this bad

This is an overly prescriptive outcome issue. These criteria are mirroring the GT too closely instead of checking only what the prompt actually requires.

The prompt does require the model to use the cancelled Amazon order as the budget source and to use the two most expensive 2025 PSU purchases to define the valid price range for the new Corsair PSU. But it does not explicitly require the final answer to report the exact cancelled-order amount or to list the names and prices of those two historical PSU purchases.

Those details may be useful intermediate reasoning, and the GT chose to surface them, but that does not make them mandatory outcome requirements. A valid trajectory could use those values correctly to filter the PSU and compute the remaining budget without explicitly reporting all of them back to the user.

The root cause is the same common mistake: the rubric is mirroring what the GT happened to say, instead of asking what the user actually needed delivered.

How to fix it

These criteria should be removed.

If a detail is only supporting reasoning used to reach the final answer, and the prompt did not explicitly ask for it to be reported, it should not be turned into a mandatory outcome criterion, although it's okay to include it in the Golden Trajectory (the info is supported and adds value to the response).

